

LLaSA: Large Language and Structured Data Assistant

Yao Xu^{1,2}, Shizhu He^{1,2}, Jiabei Chen^{1,2}, Xiangrong Zeng³,
Bingning Wang³, Jun Zhao^{1,2}, Kang Liu^{1,2,4*}

¹ The Laboratory of Cognition and Decision Intelligence for Complex Systems,
Institute of Automation, Chinese Academy of Sciences, Beijing, China

² School of Artificial Intelligence, University of Chinese Academy of Sciences, Beijing, China

³ Baichuan Inc, Beijing, China

⁴ Shanghai Artificial Intelligence Laboratory, Shanghai, China

{yao.xu, shizhu.he, jzhao, kliu}@nlpr.ia.ac.cn, chenjiabei2024@ia.ac.cn

Abstract

Structured data, such as tables, graphs, and databases, play a critical role in plentiful NLP tasks such as question answering and dialogue system. Recently, inspired by Vision-Language Models, Graph Neural Networks (GNNs) have been introduced as an additional modality into the input of Large Language Models (LLMs) to improve their performance on Structured Knowledge Grounding (SKG) tasks. However, those GNN-enhanced LLMs have the following limitations: (1) They employ diverse GNNs to model varying types of structured data, rendering them unable to uniformly process various forms of structured data. (2) The pretraining of GNNs is coupled with specific LLMs, which prevents GNNs from fully aligning with the textual space and limits their adaptability to other LLMs. To address these issues, we propose **Large Language and Structured Data Assistant (LLaSA)**, a general framework for enhancing LLMs' ability to handle structured data. Specifically, we represent various types of structured data in a unified hypergraph format, and use self-supervised learning to pretrain a hypergraph encoder, and a G-Former compressing encoded hypergraph representations with cross-attention. The compressed hypergraph representations are appended to the serialized inputs during training and inference stages of LLMs. Experimental results on multiple SKG tasks show that our pretrained hypergraph encoder can adapt to various LLMs and enhance their ability to process different types of structured data. Besides, LLaSA, with LoRA fine-tuning, outperforms previous SOTA method using full parameters tuning.

1 Introduction

Structured data, such as tables, knowledge graphs, and databases, is prevalent in real-world applications and plays a crucial role in fields like finance,

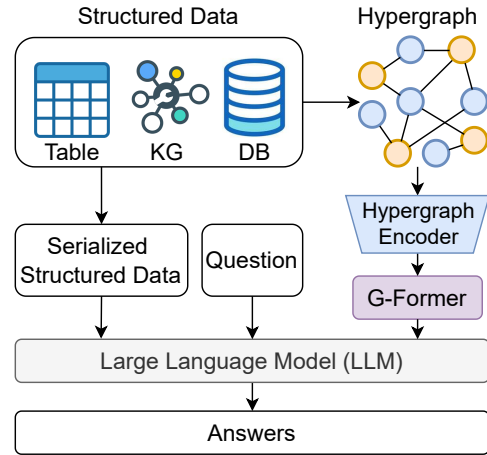


Figure 1: Overview of LLaSA, which can handle various types of structured data by transforming them into a unified format and encoding them with a universal encoder. The serialized structured data and the graph representations are then used as input to the LLM.

healthcare, and data analytics. Therefore, Structured Knowledge Grounding (SKG) (Xie et al., 2022) has attracted significant research interest and has been widely studied. SKG tasks, such as question answering (Pasupat and Liang, 2015; Nan et al., 2022; Talmor and Berant, 2018), summarization (Nan et al., 2020; Parikh et al., 2020), fact verification (Chen et al., 2019), utilizing corresponding structured data as input and produce different outputs depending on the task types.

In recent years, with the rapid development of Large Language Models (LLMs) (Bang et al., 2023; Zhao et al., 2023), researchers have shifted their focus from building task-specific models for different tasks (Xie et al., 2022) to developing a generalist model capable of handling a variety of SKG tasks (Zhuang et al., 2024; Zhang et al., 2024b). These approaches that leverage LLMs for SKG tasks commonly serialize structured data (e.g., representing tables in markdown format) as pure textual input to the LLMs. However, this method can

* Corresponding Author

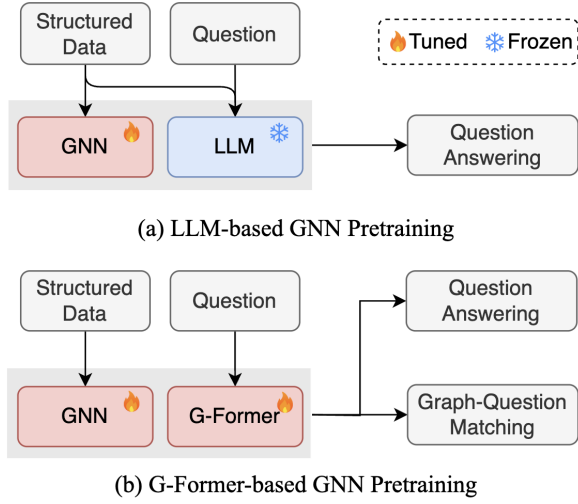


Figure 2: Comparison between LLM-based and the proposed G-Former-based GNN pretraining strategies.

lead to the partial loss of structured information, as all these LLMs are decoder-only Transformer models (Vaswani et al., 2017) (e.g., in the table data, cells from the same column or rows in the original table may become distant from each other after linear serialization).

Recently, to enhance the utilization of large language models in the visual domain, researchers have crafted Vision-Language Models (VLM) (Zhang et al., 2024a) that transform image data into discrete language tokens via a learnable interface. Inspired by the success of VLM, another line of research introduces GNNs as an additional modality into the input of LLMs. For example, G-Retriever (He et al., 2024) combines GNNs encoding knowledge graphs with LLMs, enhancing the graph-based question-answering abilities of the LLMs. HGT (Jin et al., 2024) propose a heterogeneous graph enhanced large language model for table-based question answering. However, **they employ diverse networks to model varying types of structured data, rendering them unable to uniformly process various forms of structured data**, for instance, G-Retriever and HGT can only handle graphs and tables, respectively. Besides, the pretraining of GNNs is coupled with specific LLMs in these methods, for instance, HGT pretrains a GNN based on a frozen LLM by self-supervised learning, as shown in Figure 2 (a). **This prevents the GNN from fully aligning with the textual embedding space because the serialized table is already included as input during the pretraining process, making the GNN unnecessary in this situation.** As a result, it is unclear whether

the GNN effectively encodes the table data as expected during pretraining, and the adaptability of this GNN to other LLMs also remains a question.

Aiming to address these drawbacks, we introduce Large Language and Structured Data Assistant (LLaSA) for SKG tasks. Specifically, we first model various forms of structured data, such as tables and knowledge graphs, uniformly as hypergraphs (Chen et al., 2023a), enabling the use of a unified GNN for encoding. Specifically, We treat the cells in a table as nodes, with rows and columns as hyperedges, and for graphs, we treat entities as nodes and relationships as hyperedges. We then pretrain a GNN and a G-Former (a cross attention model similar to Q-Former (Li et al., 2023), but it extracts features from GNN) with self-supervised learning which includes question answering and Graph-Text Matching, as illustrated in Figure 2 (b). This pretraining approach not only aligns the GNN with the text more effectively but also avoids coupling with a specific LLM, making it adaptable to various LLMs. During fine-tuning for downstream tasks, we use the G-Former to bridge the modality gap, transforming the encoded structured data into a fixed number of soft tokens that can be understood by LLMs, as shown in Figure 1.

Results on multiple SKG datasets, including table, knowledge graph and database, demonstrate that the proposed LLaSA significantly enhances LLM’s ability to handle these structured data. With the frozen LLM, LLaSA Llama-7B achieves an average improvement of 12% across ten datasets. With LoRA the tuned LLM, it still yields an average improvement of 0.4%. Besides, LLaSA, with LoRA fine-tuning, outperforms previous SOTA method using full parameters tuning. The codes and data are available at <https://github.com/YaoXu/LLaSA>.

The main contributions of this paper can be summarized as follows:

1. We propose LLaSA, a framework that integrates the encoded representations of structured data as an additional modality into the input of LLMs.
2. We represent various forms of structured data as hypergraphs, enabling unified encoding through a single GNN, and pretrain the GNN and G-Former with self-supervised learning.
3. Experimental results demonstrate that our pre-trained GNN can be adapted to various LLMs,

enhancing their ability to handle structured data. Furthermore, ablation studies confirm the importance of both the GNN and the pre-training process.

2 Related Work

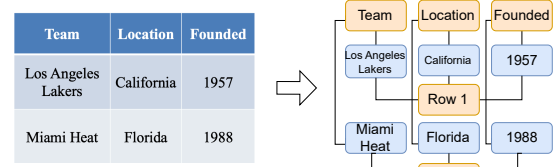
2.1 Models for SKG tasks

SKG data, such as graphs and tables, exhibit heterogeneous data formats, leading to a line of research focuses on modeling these heterogeneous representations during encoding structured data. For example, TaBERT (Yin et al., 2020) introduces vertical self-attention, a self-attention mechanism that processes vertically aligned vectors across different rows. TAPAS (Herzig et al., 2020) captures tabular structure with additional embeddings, such as Column / Row ID, based on BERT’s architecture (Devlin et al., 2019). HyTrel (Chen et al., 2023a) converts a table into a hypergraph to allow the GNN to incorporate row/column permutation invariances. All these methods can also be used in LLaSA, and we use HyTrel as our default hypergraph encoder in this work.

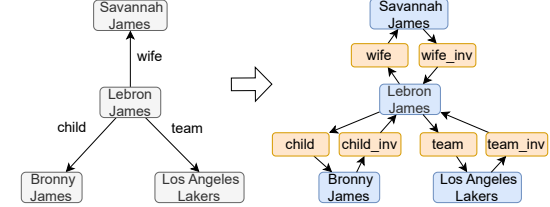
USKG (Xie et al., 2022) is the first work that unifies multiple SKG tasks into a text-to-text format. However, their results show that multi-task finetuning is worse than single-task finetuning on many tasks. Following USKG, StructLM (Zhuang et al., 2024) finetunes LLMs on multiple SKG tasks and show strong zero-shot generalization capability on unseen SKG tasks. TableLlama (Zhang et al., 2024b) finetunes LLMs with LongLoRA (Chen et al., 2023b) on multiple table-based datasets to build a generalist model. However, these methods all serialize structured data and could lead to the partial loss of structured information.

2.2 Combine LLMs and GNN

There are many works that combine LMs and GNNs (Malaviya et al., 2020; Yasunaga et al., 2022; Zhang et al., 2022; Zhao et al., 2022). In the era of LLMs, researchers are increasingly focused on how to convert GNN representations into tokens that LLMs can understand, thereby avoiding modifications to the model architecture and minimizing the impact on other capabilities. LLaGA (Chen et al., 2024) reorganizes graph nodes to structureaware sequences and then mapping these into the token embedding space through a projector. G-Retriever (He et al., 2024) uses a standard Graph Attention Network (GAT) (Veličković et al., 2017) to encode



(a) Table to Hypergraph



(b) Knowledge graph to Hypergraph

Figure 3: Examples of converting structured data to a unified hypergraph format, where yellow nodes represent hyperedges. In Figure (a), the arrows are omitted as the edges in the hypergraph are bidirectional.

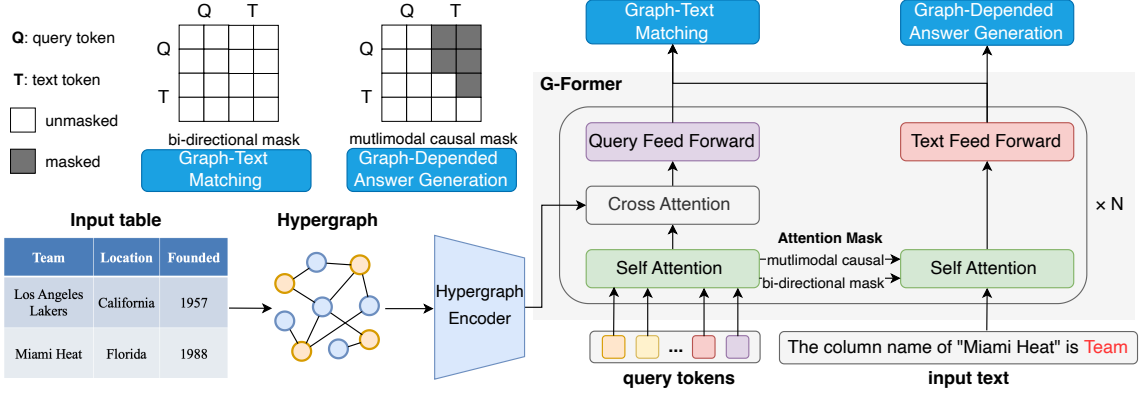
the retrieved graphs and treats graph embeddings as soft prompting, but it doesn’t involve pretraining stage. GraphGPT (Tang et al., 2023) not only appends the representations of graph nodes to the textual input, but also employs self-supervised training to align the encoding of graph structures with the natural language space. Another similar work is HGT (Jin et al., 2024), which introduces an GNN to encode the heterogeneous graph converted by the corresponding table. Both GraphGPT and HGT need to pretrain a GNN or a adapter based on a frozen LLM by self-supervised learning before task-specific instruction tuning. In contrast, our LLaSA pretrain a general GNN and G-Former that are decoupled from the specific LLM, allowing them to be used with any LLM without the need for re-pretraining, which is time-consuming.

3 Method

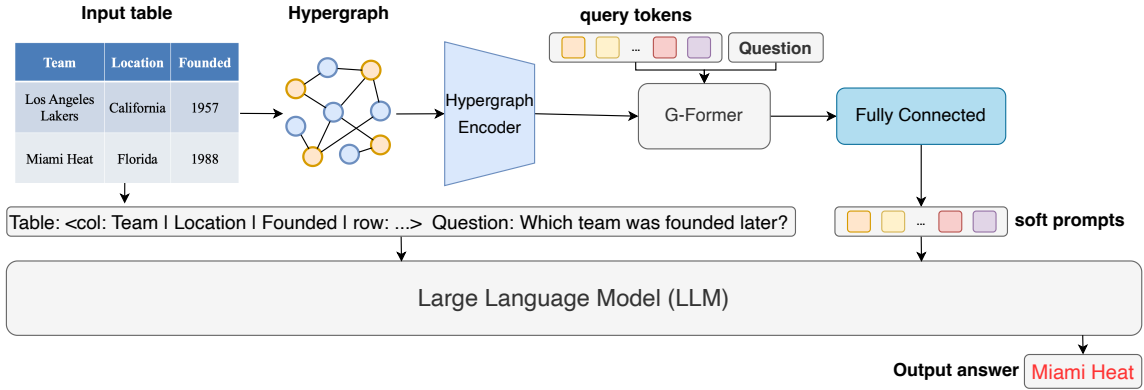
3.1 Hypergraph Construction

We represent a hypergraph as $\mathcal{G} = \{\mathcal{V}, \mathcal{E}\}$, where $\mathcal{V}$ and $\mathcal{E}$ denote the set of nodes and hyperedges. A hypergraph can be regarded as a type of bipartite graph, that is, every edge connects a node in $\mathcal{V}$ to one in $\mathcal{E}$.

Table to hypergraph. We represent a table as $\mathcal{T} = \{\mathcal{H}, \mathcal{R}\}$, where $\mathcal{H} = [h_1, h_2, \dots, h_n]$ represents n column headers, $\mathcal{R} = [r_1, r_2, \dots, r_m]$ represents m rows, and each row $r_i = [c_{i1}, c_{i2}, \dots, c_{in}]$ has n cells. We treat each cell c_{ij} as node $v_{ij} \in \mathcal{V}$, and each row r_i , each column header h_j as hyperedges



(a) Pretraining G-Former and Hypergraph encoder on template questions



(b) Finetuning LLM on SKG tasks

Figure 4: (a) We employ two pretraining objectives to train the hypergraph encoder and G-Former, with the upper left corner showing the attention masks used for different pretraining tasks. (b) In the LLM finetuning stage, we only use the graph transformer to extract a fixed number of representations of hypergraph, and treat them as soft prompts in LLM’s input.

$e_i, e_j \in \mathcal{E}$. Each node v_{ij} is only connected to its corresponding hyperedges e_i and e_j , as shown in Figure 3 (a).

Graph to hypergraph. In this work, our research focuses on Text-Attributed Graphs (TAG), which are graphs enriched with textual information associated with their nodes or edges. We represent a TAG as a set of factual triples, i.e., $G = \{(h, r, t)\}$, where h, r, t are texts, and h, t denote the head and tail entity, r denotes the relation of them. We treat h and t as nodes $v_h, v_t \in \mathcal{V}$, r as hyperedge $e_r \in \mathcal{E}$, and v_h, v_t are connected to e_r . Besides, to preserve the directional information in the original graph, we create a reverse relation node for each relation node, as shown in Figure 3 (b) (e.g., the texts of normal/reverse relation node are *Relation: team* and *Reverse Relation: team*, respectively).

3.2 Model Architecture

3.2.1 Hypergraph Encoder

Following HyTrel (Chen et al., 2023a), we utilize a HyperTrans, which is a structure-aware transformer module, to encode the hypergraphs. Each layer of HyperTrans contains two attention module: Node2Hyperedge and Hyperedge2Node, and a Hyperedge Fusion module. The initial representation of nodes are obtained by sentence bert (Reimers, 2019).

The Node2Hyperedge attention module aggregates information to hyperedge e from its neighbor nodes $v \in \mathcal{N}_e$. This process is defined as follows:

$$\tilde{\mathbf{h}}_e^{l+1} = f_{\mathcal{V} \rightarrow \mathcal{E}}(K_e^l) \quad (1)$$

where $f_{\mathcal{V} \rightarrow \mathcal{E}}$ is a attention function, $K_e^l = \{\mathbf{h}_v^l | v \in \mathcal{N}_e\}$ represents the set of representations at layer l of all nodes connected to the hypernode e .

The Hyperedge Fusion module is a Multilayer Perceptron (MLP) that integrates the information collected from both the neighbors of hypernode e and itself. This process is defined as follows:

$$\mathbf{h}_e^{l+1} = \text{MLP}(\mathbf{h}_e^l; \tilde{\mathbf{h}}_e^{l+1}) \quad (2)$$

The Hyperedge2Node attention module then aggregates information to node v from its neighbor hypernodes $e \in \mathcal{N}_v$.

$$\tilde{\mathbf{h}}_v^{l+1} = f_{\mathcal{E} \rightarrow \mathcal{V}}(K_v^l) \quad (3)$$

where $f_{\mathcal{E} \rightarrow \mathcal{V}}$ is another attention function, $K_v^l = \{\mathbf{h}_e^l | e \in \mathcal{N}_v\}$ represents the set of representations at layer l of all hypernodes connected to the node e .

The attention function f used in the equation (1, 3) is similar to transformer (Vaswani et al., 2017), the function f is defined as follows:

$$f_{\mathcal{V} \rightarrow \mathcal{E} \text{ or } \mathcal{E} \rightarrow \mathcal{V}}(\mathbf{X}) = \text{LN}(\mathbf{Y} + \text{FFN}(\mathbf{Y})) \quad (4)$$

$$\mathbf{Y} = \text{LN}(\omega + \text{SetMHA}(\omega, \mathbf{X}, \mathbf{X})) \quad (5)$$

where $\mathbf{X}$ is the representations of input nodes or hyperedges. $\mathbf{Y}$ is the intermediate representations. SetMHA is the multi-head set attention mechanism defined as follows (for simplicity, we only consider single-head self-attention here):

$$\text{SetMHA}(\omega, \mathbf{X}, \mathbf{X}) = \text{Softmax}(\omega(\mathbf{X}\mathbf{W}^K)^T(\mathbf{X}\mathbf{W}^V)) \quad (6)$$

where ω is a learnable query vector, $\mathbf{W}^K$ and $\mathbf{W}^V$ are the key and value matrices.

In summary, HyperTrans first updates the representation of a hypernode based on the neighboring nodes, and then updates the neighboring nodes using the updated representation of the hypernode.

3.2.2 G-Former

To bridge the gap between a hypergraph encoder and text, and to compress the hypergraph node representations into fixed-length tokens, we propose G-Former based on Q-Former (Li et al., 2023). As demonstrated in Figure 4 (a), our G-Former consists of two transformer sub-modules: (1) A graph transformer which interacts with hypergraph representations. (2) A text transformer that encodes and generates text. The graph transformer uses a fixed number of learnable query tokens, which first interact with each other through self-attention, then interact with hypergraph nodes representation through cross-attention. During the pretraining stage, we use different attention mechanisms based on the specific pretraining tasks to control the interaction between query and text tokens.

3.3 Training

3.3.1 Pretraining

Even though HyTrel (Chen et al., 2023a) also trains an encoder for table encoding, its pretraining tasks, such as column type classification and table similarity prediction, does not truly align the hypergraph encoder space with the textual space. Similar to Q-Former (Li et al., 2023), we also introduce two tasks to effectively align these two spaces, and their attention mechanisms are shown in the upper left corner of Figure 4. The details of constructing pretraining dataset can be founded in Appendix A. **Graph-Depended Answer Generation.** This task trains the G-Former to generate answers, given input tables as the condition. The information required for generating the answer is first extracted by the query tokens with cross-attention, and then passed to the text transformer through self-attention. The graph transformer learns to compress all the graph node representations into a fixed number of query tokens. The multimodal causal attention allow query tokens to interact with each others but not the text tokens while each text token can interact with all query tokens and its previous text tokens.

Graph-Text Matching. Since some answers can be easily deduced even without any structural information, for example, the column name for "Miami Heat" is likely to be "Team". Therefore, we also introduce Graph-Text Matching. We employ a bi-directional self-attention mask, allowing all queries and text tokens to attend to each other. As a result, the output query embeddings, denoted as Q , integrate multimodal information effectively. Each query embedding is then passed through a MLP to generate a corresponding logit. Finally, we compute the overall matching score by averaging the logits across all queries.

3.3.2 Task-specific Instruction Tuning

In the instruction tuning stage, we use multiple SKG tasks to finetune LLMs through Parameter-Efficient Fine-Tuning (Ding et al., 2023). In this stage, we only use the graph transformer module pretrained in the pretraining stage, that is, we extract fixed-length query embeddings $\hat{\mathbf{q}}$ from the node representations of hypergraph $\mathcal{G}$. Then the extracted query embeddings $\hat{\mathbf{q}}$ are projected into the same dimension as the text embedding of the LLM through a fully connected layer. This process

	Held In											Held Out			
Dataset	WikiTQ	HybridQA	FeTaQA	TabMWP	WikiSQL	TabFact	ToTTo	KVRet	CWQ	DART	Avg	SQA	WTT	FinQA	Avg
Metric	Ex	Acc	BLEU	Acc	Ex	Acc	BLEU	Micro	Acc	BLEU		Acc	BLEU	Acc	
1-shot Learning															
Mistral 7B Instruct	20.0	22.9	8.9	30.5	24.6	54.8	16.8	54.0	34.5	43.5	31.1	3.2	3.8	6.7	4.6
ChatGPT 3.5	42.6	38.4	15.1	52.9	50.4	53.0	22.4	53.0	50.1	57.0	43.5	9.7	4.0	12.2	8.6
ChatGPT 4	60.8	50.8	8.4	72.6	35.8	79.0	21.4	60.3	66.6	53.7	50.9	5.4	3.1	18.0	8.8
Full Parameters Tuning															
USKG 3B × N	49.3	59.2	36.0	-	86.0	80.8	49.0	67.9	73.3	46.7	-	0	0	0	0
FLAN-UL2 20B	54.6	61.0	35.8	-	87.3	87.1	-	-	75.9	50.4	-	70.1†	19.4†	5.9†	31.8†
StructLM 7B-M	56.8	62.6	37.5	73.5	87.0	84.6	49.8	72.2	79.9	63.2	66.7	41.9	16.7	24.6	27.7
Lora Tuning															
TableLlama 7B	35.0*	39.4*	39.0	-	50.5*	82.5	20.8*	48.7*	-	-	-	2.6	3.0	1.4	2.3
Mistral 7B Instruct	56.9	62.4	36.7	76.9	86.8	84.1	49.0	71.2	78.1	65.0	66.7	47.1	10.7	14.2	24.0
HGT 7B-M	57.2	62.4	36.6	75.8	87.0	83.8	49.2	70.8	78.3	65.4	66.7	51.0	8.2	18.0	25.7
G-Retrieve 7B-M	57.4	62.6	36.5	76.0	86.6	84.2	48.9	71.6	78.4	65.2	66.7	50.9	12.5	16.6	26.7
LlaSA 7B-M (Ours)	56.2	62.9	37.0	76.7	87.1	84.3	49.0	72.3	78.4	64.7	66.9	51.3	16.3	13.9	27.2

Table 1: The evaluation results of our model against other baselines. Cells with "*" represent that the model did not train on this dataset. Cells in the held-out section with "†" are held-in results. 7B-M represents using Mistral-7B-Instruct-v0.2 as the base model. The results of HGT 7B-M and G-Retrieve 7B-M were re-implemented by us. The results of StructLM 7B-M are from their paper. USKG 3B $\times$ N indicates training a 3B model for each task. The boldface indicates the best result.

is defined as follows:

$$\hat{\mathbf{q}} = \text{FC}(f_g(\mathbf{q}, \mathbf{X})) \quad (7)$$

where $\mathbf{X} \in \mathcal{R}^{n \times d_1}$ is the hypergraph node embeddings, n is the number of nodes in the graph, d_1 is the dimension of node embeddings, $\mathbf{q} \in \mathcal{R}^{b \times d_1}$ is the original query embeddings, $\hat{\mathbf{q}} \in \mathcal{R}^{m \times d_1}$ is the extracted query embeddings, m is the number of query tokens, d_l is the dimension of the LLM’s text embeddings, f_g and FC represents G-Former and fully connected layer.

These projected query embeddings are treated as soft prompts and appended to the text embeddings. The LLM learns to predict answers based on these text embeddings and soft prompts. This process is defined as follows:

$$\mathbf{h}_t = \text{TextEmbedder}([\text{serialize}(\mathcal{G}); x_q]) \quad (8)$$

$$p_\theta(Y|\mathcal{G}, x_q) = \prod_{i=1}^r p_\theta(y_i | y_{\leq i}, [\mathbf{h}_t; \hat{\mathbf{q}}]) \quad (9)$$

where θ is the LLM’s parameters, *serialize* denotes function that serializes structured data to text sequence, $[\cdot]$ represents concatenation operation, x_q and Y represents the question and answer, respectively.

4 Experiment

4.1 Datasets

To validate the effectiveness of our approach, we collected 10 SKG tasks as our training data, which

can be categorized into the following four types:

(1) **Structured Data Question Answering:** This task requires the LLM to answer questions based on the given tables, knowledge graphs, and textual information. The datasets for this category include WikiTQ (Pasupat and Liang, 2015), CompWebQ (Talmor and Berant, 2018), and TabMWP (Lu et al., 2022). (2) **Fact Verification:** This task requires the LLM to determine whether a given statement is *entailed* or *refuted* based on the information in the table. The corresponding dataset is TabFact (Chen et al., 2019). (3) **Structured Data to Text:** This task requires the LLM to summarize or describe the content of a given table or knowledge graph in one or two sentences. The relevant datasets for this category include ToTTo (Parikh et al., 2020) and DART (Nan et al., 2020).

To evaluate the generalization ability of our method, we use SQA (Iyyer et al., 2017), WikiTableText (Bao et al., 2018) and FinQA (Chen et al., 2021) as held-out datasets, where SQA belongs to table-based question, WikiTableText belongs to structured data to Text and FinQA requires generating python-executable math expression based on the given questions and tables.

Statistics of these datasets can be found in Appendix B.

4.2 Baselines

In this work, we compare LLaSA with other LLMs based methods. We primarily select StructLM (Zhuang et al., 2024), which performs full pa-

	Held In											Held Out			
dataset	WikiTQ	HybridQA	FeTaQA	TabMWP	WikiSQL	TabFact	ToTTo	KVRet	CWQ	DART	Avg	SQA	WTT	FinQA	Avg
metric	Ex	Acc	BLEU	Acc	Ex	Acc	BLEU	Micro	Acc	BLEU		Acc	BLEU	Acc	
Freeze LLM															
Phi 3B	31.2	39.0	13.6	41.9	47.1	59.9	32.4	46.3	45.1	58.0	41.5	14.8	8.1	9.2	10.7
LlaSA-Phi 3B	35.1	49.1	26.6	62.5	65.4	70.3	39.9	62.1	60.2	59.7	53.1	21.8	11.7	13.7	15.7
Llama2 7B	27.2	42.3	8.9	27.3	45.0	50.8	32.7	46.8	52.2	51.6	38.5	11.0	7.7	2.1	6.9
LlaSA-Llama2 7B	32.5	50.2	26.6	48.5	62.9	66.2	40.6	62.0	62.1	58.3	51.0	19.9	17.5	3.5	13.6
Mistral 7B	34.1	44.2	5.3	37.9	55.3	59.4	35.2	43.7	55.4	58.1	42.9	26.7	14.8	12.1	17.9
LlaSA-Mistral 7B	38.4	50.6	27.3	59.9	70.1	73.6	42.4	65.7	67.1	59.2	55.4	25.9	6.0	7.4	13.1
Llama3 8B	41.2	50.1	20.3	52.7	67.0	66.1	38.2	53.2	62.4	59.3	51.1	31.2	13.0	17.0	20.4
LlaSA-Llama3 8B	45.9	53.8	29.9	70.5	74.8	78.2	43.1	64.3	69.6	60.4	59.1	29.0	12.2	23.1	21.4
Lora Tuning LLM															
Phi 3B	45.8	53.6	30.7	70.0	80.2	75.5	42.6	62.5	68.3	62.9	59.2	34.3	9.5	11.4	18.4
LlaSA-Phi 3B	47.4	55.4	31.6	72.4	81.5	77.5	44.5	67.8	70.8	62.0	61.1	45.8	13.0	7.1	22.0
Llama2 7B	45.0	59.5	32.5	62.8	82.9	78.1	46.3	67.1	75.0	63.8	61.3	35.3	8.6	6.5	16.8
LlaSA-Llama2 7B	45.9	60.0	33.0	64.2	83.0	78.6	47.0	66.6	76.0	63.1	61.7	35.5	8.6	9.7	17.9
Mistral 7B	56.9	62.4	36.7	76.9	86.8	84.1	49.0	71.2	78.1	65.0	66.7	47.1	10.7	14.2	24.0
LlaSA-Mistral 7B	56.2	62.9	37.0	76.7	87.1	84.3	49.0	72.3	78.4	64.7	66.9	51.3	16.3	13.9	27.2
Llama3 8B	59.4	62.8	34.1	77.0	86.2	85.8	47.7	69.2	78.9	64.0	66.5	47.7	11.7	21.5	27.0
LlaSA-Llama3 8B	60.4	63.0	34.7	77.8	86.3	86.1	48.0	69.2	79.0	63.1	66.8	52.6	10.8	22.8	28.7

Table 2: The evaluation results of LLaSA with different base models under different finetuning strategies. The soft tokens in prompt tuning (Freeze LLM) is set 10, which is the same as the number of query tokens in G-Former. The lora rank is set to 32 in lora tuning.

rameters fine-tuning on various SKG datasets, as the main baseline. It is important to note that StructLM utilizes a broader range of datasets such as SQL2Text (Shu et al., 2021). These datasets are excluded from LLaSA’s training set because their inputs could not be transformed into hypergraphs. We also compare with TableLLama (Zhang et al., 2024b), which not only leverages a broader range of foundational table tasks, such as Column Type Annotation and Entity Linking, but also uses a longer 8K context length to finetune the LLMs. As HGT(Jin et al., 2024) and G-Retrieve(He et al., 2024) use different models and training datasets, we re-implement and train them under our framework for a fairer comparison, where HGT concatenates the representations of all GNN nodes to the LLM input, G-Retrieve takes the average of all GNN node representations and concatenate it to the LLM input. Additionally, we also evaluate the performance of GPT-3.5, GPT-4, and Mistral-7B-Instruct-v0.2 (Jiang et al., 2023) under a 1-shot setting.

4.3 Implement Details

We choose Phi-3B (Marah Abidin, 2024), LLaMA2-7B (Hugo Touvron, 2023), Mistral-7B (Jiang et al., 2023) and LLaMA3-8B (Abhimanyu Dubey, 2024) as our base models. We use a learning rate of 2e-5 with a 3% warm-up cosine scheduler, set the batch size as 3, epoch as 3. The default lora rank is set

to 32. All this models are trained on 8 H800 80G using DeepSpeed ZeRO-2 (Aminabadi et al., 2022). A training on a 7B model takes about 12 hours. The maximum sequence length is set to 2048 during training, the maximum generation length to set as 1024 during inference. We set the dimension of the hypergraph encoder to 768, with 12 layers, and use RoBERTa-base as the initial parameters for the G-Former. The total number of parameters for both components is 400M. The G-Former and GNN are pretrained on 25M tables for one epoch, and these data are taken from TaBERT (Yin et al., 2020).

4.4 Main results

Table 1 presents the results of our LLaSA compared to previous baselines across 10 datasets. From the table, we can see that GPT-3.5 and GPT-4 still fall short in handling SKG tasks, trailing behind LLaSA 7B-M by 23.3% and 15.9% points, respectively, across the ten tasks. Moreover, our LLaSA 7B-M achieves state-of-the-art (SOTA) performance in 4 out of 10 tasks within the LLM-based method.

It can be found that HGT 7B-M and G-Retrieve 7B-M did not achieve much improvement compared to the naive LLM, which may be due to the following reasons: 1) **The projector-based strategy introduces noise by feeding all node representations into the LLM.** In real-world structured data question-answering scenarios, many cells are irrelevant to the current question and may distract

the LLM. In contrast, our G-Former compresses all graph node representations into a fixed-length token sequence, retaining only the most relevant information. 2) **The projector-based strategy fails to fully leverage the alignment objectives in pre-training.** During pretraining, we aligned the Q-Former, which acts as a bridge between GNN representations and the textual space, rather than directly aligning the GNN itself. Consequently, even pretrained GNNs cannot directly enhance the final performance of the LLM.

From the perspective of model parameters, we find that the performance of LoRA-tuned Mistral 7B closely approaches that of the fully fine-tuned StructLM 7B-M. When using LLaSA framework, the LoRA-tuned Mistral 7B can surpass StructLM 7B-M, despite the former only requiring 400M trainable parameters compared to the latter’s 7B parameters, and outperforms StructLM 7B-M on 6 tasks. Additionally, we observe that fully fine-tuning LLMs on SKG tasks may lead to a decline in their performance on other tasks, whereas LoRA-tuned LLMs experience a smaller drop. One piece of evidence is the TabMWP dataset, which requires mathematical reasoning, where LLaSA 7B-M significantly outperforms StructLM 7B-M by 4.2%.

On the held-out data, although StructLM 7B achieves a higher average performance, our LLaSA significantly outperforms StructLM 7B on the SQA dataset.

4.5 LLaSA with Different Base Models

To verify the generality of our pretrained hypergraph encoder and G-Former, we evaluate LLaSA under different base models with different finetuning strategies (Prompt Tuning and Lora Tuning), the results are demonstrated in Table 2.

As shown in the table, the pretrained hypergraph encoder and G-Former enhance the models’ ability to handle SKG tasks and improve their generalization to unseen datasets across most base models (except for the prompt-tuned LLaSA-Mistral 7B, which shows a performance drop on the held-out data). Especially under the Freeze LLM setting, LLaSA achieves significant improvements compared to basic prompt tuning. Specifically, it delivers an approximate 10% performance boost across Phi-3B, Llama2-7B, Mistral-7B, and Llama3-8B models. **This indicates that our pre-trained hypergraph encoder and G-Former can be effectively adapted to various LLMs, enhancing their ability to handle structured data.**

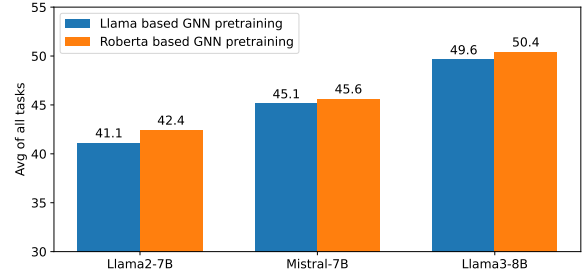


Figure 5: The average performance of models using different pretrained Hypergraph Encoder (GNN).

Under the LoRA tuning LLM setting, although LLaSA achieves smaller improvements on the held-in datasets, it consistently enhances model performance on held-out datasets. This suggests that our approach genuinely improves the model’s ability of handling structured data rather than merely overfitting to the training data. Additionally, we observed that LLaSA’s 0.9% performance improvement on Phi-3B is notably greater than the 0.3% improvement on Llama3-8B. This difference may be attributed to Phi-3B’s inherently weaker ability to process structured data, making the introduction of the hypergraph encoder more impactful in enhancing its performance.

4.6 Comparison Between Different Pretraining Strategies

We compared two strategies for pre-training the GNN: (1) Llama based pretraining, which use question answering task to pretrain a GNN based on a frozen LLM, as shown in Figure 2 (a). (2) G-Former-based pretraining, using both answer prediction and Graph-Text Matching tasks, as shown in Figure 2 (b). We select checkpoints with the same training time for both strategies and conduct experiments under the frozen LLM setting, the results are shown in Figure 5. The experimental results indicate that the GNN pretrained with G-Former exhibits superior adaptability, showing greater improvements across various models compared to the GNN pretrained with Llama. The main reasons are as follows: (1) In the Llama based pretraining, the soft tokens are not essential since serialized text is already included in the input, which prevents the GNN from fully aligning with the textual embedding space. As a result, it is unclear whether the GNN effectively encodes the table data as expected or just helps the LLMs fit the training data better. (2) The GNN pretrained with Llama primarily aligns with the Llama text space, limiting

Method	Avg-I	Avg-O	Avg
LLaSA Llama-7B	51.0	13.6	32.3
w/o pretraining	47.2	8.6	27.9
w/o GNN	42.2	7.3	24.8
w/o G-Former	42.7	7.7	25.2
prompt tuning	38.5	6.9	22.7

Table 3: Ablation results on LLaSA Llama-7B. Avg-I and Avg-O represent the average score of Held-In and Held-Out datasets. The "w/o pretraining": randomly initializing GNN and G-Former without pre-training. The "w/o GNN": ignoring cross attention in G-Former. The "w/o G-Former": ignoring the whole G-Former.

its adaptability to other models.

4.7 Ablation Study

Table 3 presents the results of our ablation study on LLaSA Llama2-7B under the frozen LLM setting. From the table, we can observe that compared to the randomly initialized GNN, the pretrained GNN helps the LLM achieve improvements of 3.8% on Held-In datasets and 5.0% on Held-Out datasets. This clearly demonstrates the effectiveness of the pretraining process.

In the "w/o GNN" and "w/o G-Former" settings, hypergraph information is ignored. The former directly passes the query token through multiple layers of self-attention, while the latter only applies a linear transformation via a fully connected layer. They can be viewed as more complex forms of prompt tuning. Although these two settings achieved a small 4% improvement on Held-In datasets compared to basic prompt tuning, they do not show significant gains on Held-Out datasets. This suggests that simple prompt tuning mainly helps the model fit the training data better, without truly enhancing its generalization capability.

5 Conclusion

In this work, we propose LLaSA, a framework that converts structured data into hypergraphs and integrates the hypergraphs representations as an additional modality into the input of LLMs. We pretrain the hypergraph encoder on 25M tables with self-supervised learning. The experimental results on different LLMs over multiple datasets demonstrate the effectiveness and generalization of our method.

Limitation

The limitations of our proposed LLaSA are as follows: (1) We used a fixed number of query tokens,

but the number of nodes in the hypergraph varies significantly, with some graphs having as few as a dozen nodes and others having over a hundred. As a result, when faced with graphs that have a large number of nodes, the G-Former may struggle to capture information effectively. (2) Due to resource constraints, we conduct our experiments using a context length of 2K instead of the 8K used in TableLlama. The performance of LLaSA in longer contexts remains to be evaluated further.

Ethics Statement

This paper proposes a method for SKG, and the experiments are conducted on public available datasets. As a result, there is no data privacy concern. Meanwhile, this paper does not involve human annotations, and there are no related ethical concerns.

Acknowledgment

This work was supported by the National Key RD Program of China (No. 2022ZD0160503) and Beijing Natural Science Foundation (L243006) and the National Natural Science Foundation of China (No.62376270).

References

- Abhinav Pandey Abhishek Kadian Ahmad Al-Dahle Aiesha Letman et al. Abhimanyu Dubey, Abhinav Jauhri. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Reza Yazdani Aminabadi, Samyam Rajbhandari, Minjia Zhang, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Jeff Rasley, Shaden Smith, Olatunji Ruwase, and Yuxiong He. 2022. [DeepSpeed inference: Enabling efficient inference of transformer models at unprecedented scale](#). *Preprint*, arXiv:2207.00032.
- Yejin Bang, Samuel Cahyawijaya, Nayeon Lee, Wenliang Dai, Dan Su, Bryan Wilie, Holy Lovenia, Ziwei Ji, Tiezheng Yu, Willy Chung, Quyet V. Do, Yan Xu, and Pascale Fung. 2023. [A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity](#). *arXiv preprint*. ArXiv:2302.04023 [cs].
- Junwei Bao, Duyu Tang, Nan Duan, Zhao Yan, Yuanhua Lv, Ming Zhou, and Tiejun Zhao. 2018. Table-to-text: Describing table region with natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32.
- Pei Chen, Soumajyoti Sarkar, Leonard Lausen, Balasubramaniam Srinivasan, Sheng Zha, Ruihong Huang,

- and George Karypis. 2023a. [HYTREL: Hypergraph-enhanced Tabular Data Representation Learning](#). *arXiv preprint*. ArXiv:2307.08623 [cs].
- Runjin Chen, Tong Zhao, Ajay Jaiswal, Neil Shah, and Zhangyang Wang. 2024. [LLaGA: Large Language and Graph Assistant](#). In *ICML 2024*. arXiv.
- Wenhu Chen, Hongmin Wang, Jianshu Chen, Yunkai Zhang, Hong Wang, Shiyang Li, Xiyu Zhou, and William Yang Wang. 2019. Tabfact: A large-scale dataset for table-based fact verification. *arXiv preprint arXiv:1909.02164*.
- Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. 2023b. Longlora: Efficient fine-tuning of long-context large language models. *arXiv preprint arXiv:2309.12307*.
- Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, et al. 2021. Finqa: A dataset of numerical reasoning over financial data. *arXiv preprint arXiv:2109.00122*.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. [BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding](#). *arXiv:1810.04805 [cs]*. ArXiv:1810.04805.
- Ning Ding, Yujia Qin, Guang Yang, Fuchao Wei, Zonghan Yang, Yusheng Su, Shengding Hu, Yulin Chen, Chi-Min Chan, Weize Chen, et al. 2023. Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3):220–235.
- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. 2024. [G-Retriever: Retrieval-Augmented Generation for Textual Graph Understanding and Question Answering](#). *arXiv preprint*. ArXiv:2402.07630 [cs].
- Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisenschlos. 2020. [TaPas: Weakly Supervised Table Parsing via Pre-training](#). In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 4320–4333, Online. Association for Computational Linguistics.
- Kevin Stone Peter Albert Amjad Almahairi-Yasmine Babaei Nikolay Bashlykov et al. Hugo Touvron, Louis Martin. 2023. [Llama 2: Open foundation and fine-tuned chat models](#). *Preprint*, arXiv:2307.09288.
- Mohit Iyyer, Wen-tau Yih, and Ming-Wei Chang. 2017. Search-based neural structured learning for sequential question answering. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1821–1831.
- Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, L  lio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timoth  e Lacroix, and William El Sayed. 2023. [Mistral 7b](#). *Preprint*, arXiv:2310.06825.
- Rihui Jin, Yu Li, Guilin Qi, Nan Hu, Yuan-Fang Li, Jiaoyan Chen, Jianan Wang, Yongrui Chen, and Dehai Min. 2024. [HGT: Leveraging Heterogeneous Graph-enhanced Large Language Models for Few-shot Complex Table Understanding](#). *arXiv preprint*. ArXiv:2403.19723 [cs].
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. 2023. [BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models](#). *arXiv preprint*. ArXiv:2301.12597 [cs].
- Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. 2022. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. *arXiv preprint arXiv:2209.14610*.
- Chaitanya Malaviya, Chandra Bhagavatula, Antoine Bosselut, and Yejin Choi. 2020. Commonsense knowledge base completion with structural and semantic context. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 2925–2933.
- Hany Awadalla Ahmed Awadallah Ammar Ahmad Awan Nguyen Bach et al. Marah Abdin, Jyoti Aneja. 2024. [Phi-3 technical report: A highly capable language model locally on your phone](#). *Preprint*, arXiv:2404.14219.
- Linyong Nan, Chiachun Hsieh, Ziming Mao, Xi Victoria Lin, Neha Verma, Rui Zhang, Wojciech Kry  ci  ski, Hailey Schoelkopf, Riley Kong, Xiangru Tang, et al. 2022. Fetaqa: Free-form table question answering. *Transactions of the Association for Computational Linguistics*, 10:35–49.
- Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna, et al. 2020. Dart: Open-domain structured data record to text generation. *arXiv preprint arXiv:2007.02871*.
- Ankur P Parikh, Xuezhi Wang, Sebastian Gehrmann, Manaal Faruqui, Bhuwan Dhingra, Diyi Yang, and Dipanjan Das. 2020. Totto: A controlled table-to-text generation dataset. *arXiv preprint arXiv:2004.14373*.
- Panupong Pasupat and Percy Liang. 2015. Compositional semantic parsing on semi-structured tables. *arXiv preprint arXiv:1508.00305*.
- N Reimers. 2019. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*.

- Chang Shu, Yusen Zhang, Xiangyu Dong, Peng Shi, Tao Yu, and Rui Zhang. 2021. Logic-consistency text generation from semantic parses. *arXiv preprint arXiv:2108.00577*.
- Alon Talmor and Jonathan Berant. 2018. The web as a knowledge-base for answering complex questions. *arXiv preprint arXiv:1803.06643*.
- Jiabin Tang, Yuhao Yang, Wei Wei, Lei Shi, Lixin Su, Suqi Cheng, Dawei Yin, and Chao Huang. 2023. [GraphGPT: Graph Instruction Tuning for Large Language Models](#). arXiv. ArXiv:2310.13023 [cs].
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. [Attention Is All You Need](#). *arXiv preprint*. ArXiv:1706.03762 [cs].
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2017. Graph attention networks. *arXiv preprint arXiv:1710.10903*.
- Tianbao Xie, Chen Henry Wu, Peng Shi, Ruiqi Zhong, Torsten Scholak, Michihiro Yasunaga, Chien-Sheng Wu, Ming Zhong, Pengcheng Yin, Sida I. Wang, Victor Zhong, Bailin Wang, Chengzu Li, Connor Boyle, Ansong Ni, Ziyu Yao, Dragomir Radev, Caiming Xiong, Lingpeng Kong, Rui Zhang, Noah A. Smith, Luke Zettlemoyer, and Tao Yu. 2022. [Unified-SKG: Unifying and Multi-Tasking Structured Knowledge Grounding with Text-to-Text Language Models](#). *arXiv preprint*. ArXiv:2201.05966 [cs].
- Michihiro Yasunaga, Antoine Bosselut, Hongyu Ren, Xikun Zhang, Christopher D Manning, Percy S Liang, and Jure Leskovec. 2022. Deep bidirectional language-knowledge graph pretraining. *Advances in Neural Information Processing Systems*, 35:37309–37323.
- Pengcheng Yin, Graham Neubig, Wen-tau Yih, and Sebastian Riedel. 2020. [TaBERT: Pretraining for Joint Understanding of Textual and Tabular Data](#). *ACL 2020*. ArXiv: 2005.08314.
- Jingyi Zhang, Jiaying Huang, Sheng Jin, and Shijian Lu. 2024a. Vision-language models for vision tasks: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- Tianshu Zhang, Xiang Yue, Yifei Li, and Huan Sun. 2024b. [TableLlama: Towards Open Large Generalist Models for Tables](#). *arXiv preprint*. ArXiv:2311.09206 [cs].
- Xikun Zhang, Antoine Bosselut, Michihiro Yasunaga, Hongyu Ren, Percy Liang, Christopher D Manning, and Jure Leskovec. 2022. Greaselm: Graph reasoning enhanced language models for question answering. *arXiv preprint arXiv:2201.08860*.
- Jianan Zhao, Meng Qu, Chaozhuo Li, Hao Yan, Qian Liu, Rui Li, Xing Xie, and Jian Tang. 2022. Learning on large-scale text-attributed graphs via variational inference. *arXiv preprint arXiv:2210.14709*.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*.
- Alex Zhuang, Ge Zhang, Tianyu Zheng, Xinrun Du, Junjie Wang, Weiming Ren, Stephen W. Huang, Jie Fu, Xiang Yue, and Wenhui Chen. 2024. [StructLM: Towards Building Generalist Models for Structured Knowledge Grounding](#). *arXiv preprint*. ArXiv:2402.16671 [cs].

Dataset	Overall Length		Train					Test				
	Input (avg)	Output (avg)	Count	Input (max)	Output (max)	# Nodes (avg)	# Trunc	Count	Input (max)	Output (max)	# Nodes (avg)	# Trunc
TabMWP	208	5	23059	709	33	20	0	7686	703	31	19	0
ToTTo	252	31	120761	2040	155	110	467	7700	2048	119	111	31
KVRet	573	17	6288	1217	161	57	0	807	1147	82	56	0
HybridQA	700	7	62682	2047	91	92	200	3466	2048	79	93	6
CompWebQ	1350	12	27639	2047	321	265	321	2816	2048	256	264	8
TabFact	660	5	92283	2045	5	94	2	12779	1687	4	93	0
WikiTQ	832	6	11321	2028	273	114	0	4344	2048	148	115	10
WikiSQL	689	7	56355	2047	518	96	16	15878	2048	244	98	1
FeTaQA	653	39	7326	1853	158	97	0	2003	1548	114	95	0
DART	134	30	62659	406	258	17	0	5097	261	109	17	0
SQA	657	35	12275	1812	1012	98	2	3011	1725	769	102	0
WikiTableText	150	27	10000	313	97	13	0	2000	226	89	14	0
Finqa	1230	21	6251	2040	72	29	186	1147	2048	61	31	25

Table 4: The statistics of numbers of input and output tokens in the training and test sets for each task. "# Trunc" indicates the number of samples where the input length exceeds 2048 tokens and has been truncated. "# Nodes" indicates the average number of nodes in hypergraphs.

A Pretraining Dataset

We use the 25 million tables collected by TaBERT (Yin et al., 2020). We designed three types of question templates and used them to generate 10 questions for each table. The specific templates are as follows:

1. *What's the column name of "{node_name}" ?*
2. *In the row where the value of {first_col_name} is "{row_value}", what is the corresponding value of {col_name}?*
3. *Are "{node_name1}" and "{node_name2}" in the same row?.*

B SKG Datasets

Some datasets are not used in our study, such as FEVEROUS and Infotabs, because the tables in these datasets are not well-structured, with some rows having a different number of cells than the table headers. The statistics of the SKG datasets we used are shown in Table 4.